

ASSIGNMENT: SE – OVERVIEW OF IT INDUSTRY

Session 1 — IT Industry Basics

1. Name 5 IT job roles and what they do

IT Job Role	What they do
1. Software Developer	Develops, tests, and maintains software applications.
2. Web Developer	Creates and maintains websites and web applications.
3. Database Administrator (DBA)	Manages databases, including data storage, security, backup, and performance.
4. Software Tester / QA Engineer	Tests software to find bugs and ensures the application works correctly.
5. DevOps Engineer	Manages software deployment, servers, automation, and CI/CD processes.

2. Product Companies vs Service Companies

➤ Product Companies

Company	Example of Work
Microsoft	Develops products such as Windows, Microsoft 365, and Azure.
Google	Develops products such as Google Search, YouTube, and Android.
Adobe	Develops software products such as Photoshop and Acrobat.

➤ Service Companies

Company	Example of Work
TCS	Provides IT consulting and software development services to clients.
Infosys	Provides IT consulting, software development, and digital services.
Wipro	Provides IT services, consulting, and technology solutions to businesses.

➤ Easy difference:

- **Product company** → Creates and sells its own software/product.
- **Service company** → Creates or manages software for clients/customers.

3. Full forms and one-line use

Term	Full Form	One-line Use
GitHub	GitHub is not an acronym; it is a platform name	Used to host, manage, and collaborate on source code using Git.
Slack	Slack is a product/platform name, not a standard full form	Used for team communication and collaboration.
Jira	Jira is a product name, not an acronym	Used to track tasks, bugs, and software development projects.
IDE	Integrated Development Environment	Used to write, run, debug, and manage code.
API	Application Programming Interface	Allows different software applications to communicate with each other.

4. Five programming languages and popular uses

Programming Language	Popular Use Case
Java	Enterprise applications and backend development.
JavaScript	Web development and interactive websites.
Python	AI, Machine Learning, data science, and backend development.
C	System programming and embedded systems.
C++	Game development and high-performance applications.

5. Developer vs Tester

- **Developer:** Writes and maintains the code to build software and features.
 - **Tester:** Tests the software to find bugs and checks whether it works according to requirements.
-

Session 2 — Software Architecture & Applications

1. Three main layers of software architecture

- **Presentation Layer** – The user interface where users interact with the application.
- **Business Logic Layer** – Contains the application rules, logic, and processing.
- **Data Layer** – Handles storing, retrieving, and managing data in the database.
- **Simple flow:**
User → Presentation Layer → Business Logic Layer → Data Layer → Database

2. Client and Server

- **Client:** A device or application that sends a request for a service or data.
- **Server:** A computer or system that receives the request, processes it, and sends a response.
- **Real-life example:**
When you open YouTube on your phone, your phone is the client, and YouTube's server sends videos and other data back to your phone.

3. Desktop Apps vs Web Apps vs Mobile Apps

Type	Difference 1	Difference 2
Desktop App	Installed directly on a computer/laptop.	Usually designed for Windows, macOS, or Linux.
Web App	Runs inside a web browser.	Usually does not require installation; accessed through a URL.
Mobile App	Designed for smartphones/tablets.	Usually installed from an app store such as Google Play or App Store.

- **Examples:**
 - Desktop → Microsoft Word
 - Web → Gmail in a browser
 - Mobile → WhatsApp mobile app

4. Three databases and their type

Database	Type
MySQL	SQL
PostgreSQL	SQL
MongoDB	NoSQL

- **Easy way to remember:**
 - **SQL** → Data is generally stored in tables (rows and columns).
 - **NoSQL** → Data can be stored in formats such as documents, key-value pairs, or graphs.
-

Session 3 — Networking Basics

1. Full forms and one-line use

Protocol	Full Form	One-line Use
HTTP	HyperText Transfer Protocol	Used to transfer web pages and data between a web browser and a web server.
HTTPS	HyperText Transfer Protocol Secure	Used to securely transfer data between a browser and server using encryption.
FTP	File Transfer Protocol	Used to upload and download files between computers over a network.
SMTP	Simple Mail Transfer Protocol	Used to send emails from a mail client to a mail server or between mail servers.

2. Difference between IPv4 and IPv6

- IPv4 uses a 32-bit address, such as 192.168.1.1, and provides about 4.3 billion addresses.
- IPv6 uses a 128-bit address, such as 2001:db8::1, and provides a much larger number of addresses.

3. nslookup google.com

- Run this command in Command Prompt (CMD):
`nslookup google.com`
- The IP address can vary depending on your location, DNS server, and time. You may get output similar to:

Name: `google.com`

Addresses: `142.250.195.14`

Note: Write the IP address shown on your own system, because google.com can return different IP addresses.

4. HTTP Status Codes

Code	Meaning	Explanation
200	OK	Request was successful.
301	Moved Permanently	The requested resource has permanently moved to a new URL.
404	Not Found	The requested page/resource does not exist.
500	Internal Server Error	An unexpected error occurred on the server.
403	Forbidden	The server understood the request but refuses to allow access.

5. DNS Hierarchy using www.google.com

- DNS (Domain Name System) converts domain names into IP addresses.

For www.google.com:

Root (.)



TLD (.com)



Domain (google)



Subdomain (www)

- **Root (.)** → The top level of the DNS hierarchy.
 - **TLD (.com)** → Top-Level Domain; .com is the TLD here.
 - **Domain (google)** → The main registered domain name.
 - **Subdomain (www)** → A subdomain used to access Google's web service.
 - **Simple example:** www.google.com = www (subdomain) + google (domain) + .com (TLD) + root (.).
-

Session 4 — SDLC, Agile, Waterfall & Scrum

1. Six phases of SDLC in order

- **SDLC** = Software Development Life Cycle
 1. **Planning** – Identify the project goals, requirements, resources, and timeline.
 2. **Requirement Analysis** – Collect and analyze what the customer needs from the software.
 3. **Design** – Create the software architecture, database design, UI design, and technical structure.
 4. **Development / Implementation** – Developers write the code and build the software.
 5. **Testing** – Test the software to find and fix bugs and ensure it meets requirements.
 6. **Deployment & Maintenance** – Release the software to users and maintain or update it after release.
- **Flow:**

Planning → Requirement Analysis → Design → Development → Testing → Deployment & Maintenance

2. Three differences between Agile and Waterfall

Agile	Waterfall
Development is iterative and incremental.	Development follows a fixed sequential process.
Requirements can change during development.	Requirements are generally defined early and changes are difficult.
Testing happens continuously during development.	Testing mainly happens after development is completed.

Easy example:

- **Agile** = Build → Test → Get feedback → Improve → Repeat
- **Waterfall** = Requirements → Design → Development → Testing → Deployment

3. Three main Scrum roles

- **Product Owner** – Manages the product requirements and prioritizes the Product Backlog.
- **Scrum Master** – Helps the Scrum team follow Scrum practices and removes obstacles.
- **Developers** – Design, develop, test, and deliver the product increment.

4. What is a Sprint in Scrum?

- A **Sprint** is a fixed period in Scrum during which the team works to complete a selected set of tasks and create a usable product increment.
- **Typical duration:** 1 to 4 weeks, with 2 weeks being very common.

5. Jira Board Columns

Column	Meaning
To Do	Tasks that are planned but work has not started yet.
In Progress	Tasks that the team is currently working on.
Done	Tasks that have been completed according to the project's definition of done.
Backlog	A list of pending tasks, requirements, bugs, and future work that has not yet been selected for active work.

- **Simple flow:**

Backlog → To Do → In Progress → Done

Session 5 — Git & GitHub

1. Git Commands

Task	Git Command
Initialize a repository	git init
Stage all files	git add .
Commit with a message	git commit -m "Initial commit"
Push to remote	git push origin main
Pull latest changes	git pull origin main

2. Difference between git fetch and git pull

- git fetch downloads the latest changes from the remote repository but does not merge them into your current branch.
- git pull downloads the latest changes and merges them into your current branch.

3. Create GitHub Repository and Push a Text File

You can do this using the following steps:

Step 1: Create a repository on GitHub

1. Open GitHub.
2. Click New repository.
3. Enter a repository name, for example: git-practice.
4. Click Create repository.

Step 2: Create a text file locally

- Create a file called:
`hello.txt`
- Put some text inside it, for example:

`Hello, GitHub!`

Step 3: Run these commands in the folder

`git init`

`git add .`

`git commit -m "Add hello text file"`

`git branch -M main`

`git remote add origin https://github.com/YOUR_USERNAME/git-practice.git`

`git push -u origin main`

- Replace YOUR_USERNAME with your GitHub username.

- **Repository link format:**

https://github.com/YOUR_USERNAME/git-practice

- I can't create the repository in your GitHub account or provide your actual repo link without access to your account, so after you push it, submit your generated GitHub repository URL.

4. Fork vs Pull Request

- **Fork:** A fork creates your own copy of another person's GitHub repository in your GitHub account.
- **Pull Request (PR):** A pull request asks the original repository owner to review and merge your changes into their repository.

5. Create feature-test Branch, Commit, and Merge

Step 1: Create and switch to the new branch

```
git checkout -b feature-test
```

Step 2: Make a change

For example, create or modify `feature.txt`.

Step 3: Stage and commit the change

```
git add .
```

```
git commit -m "Add feature test"
```

Step 4: Switch back to main

```
git checkout main
```

Step 5: Merge the branch

```
git merge feature-test
```

Complete command flow

```
git checkout -b feature-test
```

```
git add .
```

```
git commit -m "Add feature test"
```

```
git checkout main
```

```
git merge feature-test
```

Flow:

main → feature-test → commit → main → merge feature-test
